Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»
—
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 4

«Пользовательские программы. Аргументы командной строки»

по дисциплине «Операционные системы»

Выполнил
студент гр. 5151003/40001 Корпусов Д.Н

<подпись>

Руководитель Югай П.Э

<подпись>

Санкт-Петербург
2025

Содержание

Цель работы 3
Ход работы 3-10
1. Теоретическая часть 3-5
2. Практическая часть 5-11
Вывод 12
Листинг А «thread.h» 13-16
Листинг Б «process.c» 17-31
Листинг В «syscall.c» 32-33
Листинг Г «svconv.c» 34-36

ЦЕЛЬ РАБОТЫ

Цель лабораторной работы – изучение механизмов передачи параметров пользовательским программам и реализация такого механизма в архитектуре 80x86 с использованием стека.

ХОД РАБОТЫ

1. Теоретическая часть

В операционных системах пользовательские программы выполняются в режиме ограниченных привилегий и не имеют прямого доступа к ресурсам ядра. Поэтому взаимодействие между пользовательским уровнем и системой строится через контролируемые механизмы, среди которых центральное место занимают системные вызовы и правильная организация пользовательского стека при запуске процесса. Именно эти аспекты лежат в основе выполнения лабораторной работы.

Одним из ключевых элементов запуска программы является обработка аргументов командной строки. Пользователь передает строку, содержащую имя программы и параметры, которые необходимо разделить на отдельные токены. В операционных системах командная строка передается дочернему процессу уже разобранной на аргументы, поэтому ядро должно выполнить разбиение строки, проверить длины и количество аргументов, а затем передать их программе в соответствии со стандартным соглашением вызовов.

Следующим важным этапом является формирование пользовательского стека. Стек начального пользовательского процесса должен содержать копии строк аргументов, массив указателей argv[], значение argc, а также фиктивный адрес возврата. Формирование стека должно учитывать требования к выравниванию данных и порядок размещения информации — аргументы располагаются сверху вниз, далее формируется массив указателей, после чего на стек помещается количество аргументов. Правильная структура стека гарантирует корректный запуск

программы и соответствие стандарту передачи параметров на архитектуре x86.

Неотъемлемой частью выполнения программы является механизм системных вызовов. Пользовательский код не может напрямую обращаться к файловой системе или памяти ядра, поэтому все такие операции выполняются через контролируемый переход из режима пользователя в режим ядра. Обычно это реализуется через программное прерывание или специальную инструкцию, по которой ядро получает номер системного вызова и параметры, проверяет их корректность, выполняет соответствующую функцию и возвращает результат обратно. Ключевым моментом является валидация пользовательских указателей, так как ядро должно защититься от обращений к недопустимым адресам.

Завершение пользовательского процесса сопровождается передачей в ядро кода завершения, который затем должен быть доступен родительскому процессу. Для этого система поддерживает структуры, связывающие процессы между собой: родитель может ожидать завершения потомка и получить его статус. Корректная реализация этого механизма необходима для синхронизации процессов и определения успешности выполнения программы.

Таким образом, выполнение пользовательской программы в операционной системе основано на нескольких фундаментальных механизмах: разборе аргументов командной строки, построении стека начального процесса, реализации системных вызовов и передаче кода завершения от потомка к родителю. Эти компоненты обеспечивают корректную работу пользовательских программ, безопасное взаимодействие с ядром и предсказуемое управление процессами.

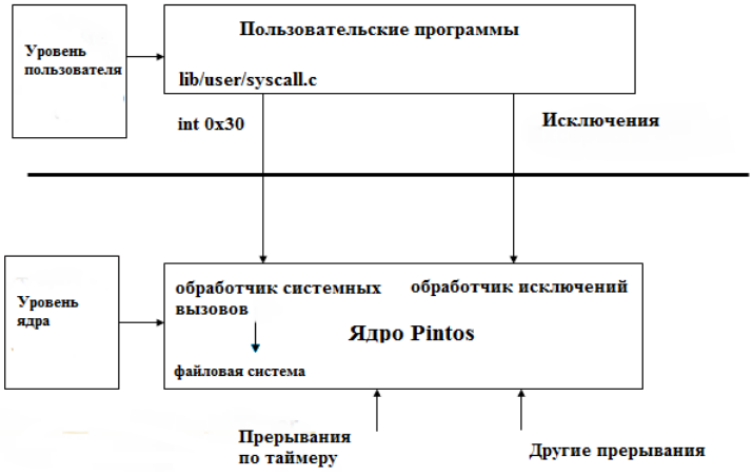


Рис. 1 «Уровни взаимодействия с ОС Pintos»

2. Практическая часть

Для реализации поддержки пользовательских процессов и передачи аргументов командной строки были расширены стандартные структуры ядра Pintos. Основные изменения касаются структуры struct thread, хранящей информацию о текущем выполняемом процессе.

Поле	Назначение
parent	Указатель на родительский процесс. Позволяет организовать связь родитель–потомок.
children	Список дочерних процессов, используемый для поиска и ожидания потомков.
child_status	Структура состояния процесса для синхронизации и передачи кода завершения.
exit_status	Код завершения процесса, возвращаемый через process_wait().
prog_name[64]	Имя исполняемой программы для вывода и диагностики.

В начале process.c была добавлена структура struct child_status. Она представляет собой служебную структуру, используемую для управления отношениями между родительским и дочерним процессами. Она создаётся в момент запуска нового процесса через process_execute() и хранится в списке детей родителя. Эта структура обеспечивает корректную работу системного вызова wait(), позволяя родительскому процессу отслеживать состояние потомков и получать их код завершения.

Первой задачей стала организация корректного запуска пользовательских программ. В функцию process_execute() были внесены изменения, позволяющие разделять командную строку на имя исполняемого файла и аргументы, копировать строку в выделенную страницу памяти, создавать структуру child_status для нового процесса и связывать её с родительским потоком. Также был реализован механизм передачи аргументов в start_process(), где формируется стек пользовательского процесса и производится вызов загрузчика ELF-файлов.

Вторая важная часть работы — реализация системных вызовов. В модуле syscall.c был написан обработчик syscall_handler(), который получает номер системного вызова, извлекает аргументы из стека пользователя и выполняет соответствующую операцию. Были реализованы вызовы exit, write и добавлена проверка корректности пользовательских указателей через функцию validate_user_pointer(), которая предотвращает аварийное завершение ядра при обращении к недоступной памяти.

Также была полностью реализована логика завершения процессов. В функции process_exit() добавлено сохранение статуса завершения в структуру child_status, установка флагов завершённости, пробуждение родителя с помощью семафора и корректное уничтожение таблицы страниц процесса. В функции process_wait() реализовано ожидание дочернего процесса: поиск нужной записи в списке детей, блокировка до завершения ребёнка, получение статуса выхода и освобождение структуры child_status.

Таким образом, в ходе работы была реализована полноценная модель пользовательских процессов: запуск программ, передача аргументов, системные вызовы, завершение процессов и механизм ожидания потомков. Все внесённые изменения обеспечивают корректное исполнение пользовательских программ и прохождение тестов Pintos для части userprog.

Была построена диаграмма состояний ожидания, передачи аргументов и результатов выполнения между основными функциями (thread_create, run_task, process_execute, process_wait, start_process. thread_exit). (см. рис. 2)

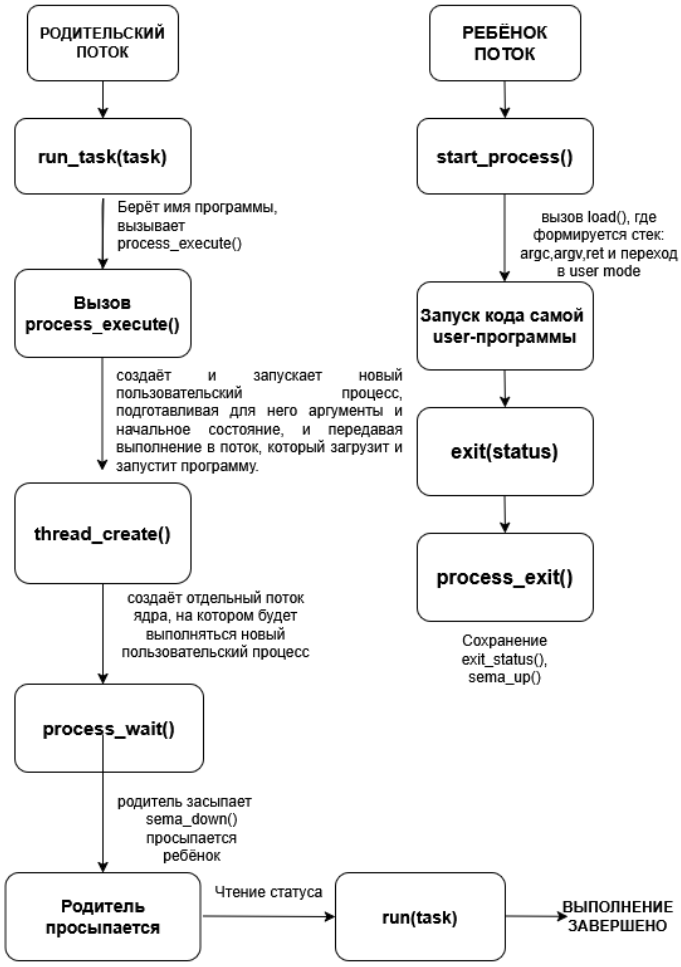


Рис. 2 «Диаграмма»

Тестовые программы серии *args-* предназначены для проверки корректности разборки командной строки и правильного формирования пользовательского стека в Pintos. Каждая из тестовых программ передает в систему определённую командную строку, содержащую имя исполняемого файла и набор аргументов, которые должны быть корректно разобраны в функции load() и переданы в пользовательскую программу через параметры argc и argv. При запуске, тестовый механизм Pintos формирует строку вида "args-single onearg" или "args-multiple one two three", после чего эта строка попадает в process_execute() и далее в load(), где происходит её разбор с использованием strtok_r. Именно там формируется массив указателей argv[], подсчитывается количество аргументов и далее подготавливается стек пользовательского процесса согласно требованиям ABI.

Тест **args-none** проверяет случай, когда программа запускается без аргументов, например командной строкой args-none. В этом случае ожидается, что пользовательская программа получит argc = 1 и ровно один элемент в argv — имя программы. Обработка этого случая происходит корректно, так как strtok_r выделяет только один токен, а стек формируется с единственным указателем и завершающим NULL. (см.рис. 3)

```
Executing 'args-none':
(args) begin
(args) argc = 1
(args) argv[0] = 'args-none'
(args) argv[1] = null
(args) end
args-none: exit(0)
Execution of 'args-none' complete.
```

Рис. 3 «Тест args-none»

В тесте **args-single** проверяется базовый случай передачи одного аргумента. Командная строка имеет вид args-single onearg, и ожидается, что пользовательская программа получит argc = 2 и корректные указатели на строки "args-single" и "onearg". Наша реализация корректно разбивает строку на два токена, копирует их на стек, формирует массив указателей и

записывает терминальный NULL, что обеспечивает правильное поведение программы при доступе к argv[2]. (см. рис. 4)

```
Executing 'args-single onearg':
(args) begin
(args) argc = 2
(args) argv[0] = 'args-single'
(args) argv[1] = 'onearg'
(args) argv[2] = null
(args) end
args-single: exit(0)
Execution of 'args-single onearg' complete.
```

Рис.4 «Тест args-single»

Тест **args-multiple** проверяет корректность обработки нескольких аргументов — строка содержит несколько слов, разделённых пробелами. Ожидается, что все аргументы будут правильно выделены, размещены в памяти и доступны в argv. Наш код использует цикл с strtok_r, который стабильно обрабатывает любое количество пробелов между аргументами, после чего стек формируется в обратном порядке, с учётом всех указателей и необходимых выравниваний. В результате программа получает точные значения argc и argv[i]. (см. рис. 5)

```
Executing 'args-multiple some arguments for you!':
(args) begin
(args) argc = 5
(args) argv[0] = 'args-multiple'
(args) argv[1] = 'some'
(args) argv[2] = 'arguments'
(args) argv[3] = 'for'
(args) argv[4] = 'you!'
(args) argv[5] = null
(args) end
args-multiple: exit(0)
Execution of 'args-multiple some arguments for you!' complete.
```

Рис. 5 «Тест args-multiple»

Тест **args-many** проверяет способность системы работать с большим количеством аргументов (до нескольких десятков). Основной целью является подтверждение того, что механизм передачи аргументов не выходит за границы допустимого размера массива, корректно работает с большим числом строк и формирует стек с правильным выравниванием. В нашей реализации предусмотрена защита от переполнения (argc < 64), а механизм

упаковки аргументов на стек и выравнивания адресов обеспечивает корректную передачу всех аргументов пользовательской программе.(см. рис. 6)

```
Executing 'args-many a b c d e f g h i j k l m n o p q r s t u v':
(args) begin
(args) argc = 23
(args) argv[0] = 'args-many'
(args) argv[1] = 'a'
(args) argv[2] = 'b'
(args) argv[3] = 'c'
(args) argv[4] = 'd'
(args) argv[5] = 'e'
(args) argv[6] = 'f'
(args) argv[7] = 'g'
(args) argv[8] = 'h'
(args) argv[9] = 'i'
(args) argv[10] = 'j'
(args) argv[11] = 'k'
(args) argv[12] = 'l'
(args) argv[13] = 'm'
(args) argv[14] = 'n'
(args) argv[15] = 'o'
(args) argv[16] = 'p'
(args) argv[17] = 'q'
(args) argv[18] = 'r'
(args) argv[19] = 's'
(args) argv[20] = 't'
(args) argv[21] = 'u'
(args) argv[22] = 'v'
(args) argv[23] = null
(args) end
args-many: exit(0)
Execution of 'args-many a b c d e f g h i j k l m n o p q r s t u v' complete.
```

Рис. 6 «Тест args-many»

Тест **args-dbl-space** проверяет корректность обработки нескольких последовательных пробелов между аргументами. Командная строка вида args-dbl-space a b c содержит двойные и тройные пробелы, однако функция strtok_r по стандарту пропускает повторяющиеся разделители, поэтому аргументы выделяются правильно. Наша реализация полностью соответствует этому требованию: пустые токены не создаются, а стек содержит корректный набор строк "a", "b" и "c". (см. рис. 7)

```
Executing 'args-dbl-space two  spaces!':
(args) begin
(args) argc = 3
(args) argv[0] = 'args-dbl-space'
(args) argv[1] = 'two'
(args) argv[2] = 'spaces!'
(args) argv[3] = null
(args) end
args-dbl-space: exit(0)
Execution of 'args-dbl-space two  spaces!' complete.
```

Рис. 7 «Тест args-dbl-space»

Также было выполнено дополнительное задание и была реализована программа svconv, которая переводит числа из одной системы счисления в другую.

Поддерживаемые системы счисления: двоичная, десятичная, восьмеричная, шестнадцатеричная.

Аргументы: число, исходная система счисления, требуемая система счисления.

Примеры работы программы:

```
Kernel command line: run 'snconv A 16 10'
Pintos booting with 3,968 kB RAM...
367 pages available in kernel pool.
367 pages available in user pool.
Calibrating timer...  419,430,400 loops/s.
hda: 1,008 sectors (504 kB), model "QM00001", serial "QEMU HARDDISK"
hda1: 192 sectors (96 kB), Pintos OS kernel (20)
hdb: 5,040 sectors (2 MB), model "QM00002", serial "QEMU HARDDISK"
hdb1: 4,096 sectors (2 MB), Pintos file system (21)
filesys: using hdb1
Boot complete.
Executing 'snconv A 16 10':
10
snconv: exit(0)
```

Рис. 8 «svconv A 16 10»

```
Kernel command line: run 'snconv 1010 2 16'
Pintos booting with 3,968 kB RAM...
367 pages available in kernel pool.
367 pages available in user pool.
Calibrating timer...  196,403,200 loops/s.
hda: 1,008 sectors (504 kB), model "QM00001", serial "QEMU HARDDISK"
hda1: 192 sectors (96 kB), Pintos OS kernel (20)
hdb: 5,040 sectors (2 MB), model "QM00002", serial "QEMU HARDDISK"
hdb1: 4,096 sectors (2 MB), Pintos file system (21)
filesys: using hdb1
Boot complete.
Executing 'snconv 1010 2 16':
A
```

Рис. 9 «svconv 1010 2 16»

ВЫВОД

В ходе выполнения лабораторной работы была реализована базовая поддержка пользовательских процессов в операционной системе Pintos. Одним из ключевых результатов стало создание механизма запуска пользовательских программ, который включает разбор командной строки, подготовку аргументов, формирование корректного пользовательского стека и запуск новой программы в отдельном потоке. Важной частью работы стала реализация обработки системных вызовов, которые обеспечивают взаимодействие пользовательского кода с ядром.

Листинг А «thread.h»

```
#ifndef THREADS_THREAD_H
#define THREADS_THREAD_H

#include <debug.h>
#include <list.h>
#include <stdint.h>

#ifdef USERPROG
struct child_status; // Существует такая структура (process.c)
#endif

/* States in a thread's life cycle. */
enum thread_status
  {
    THREAD_RUNNING,     /* Running thread. */
    THREAD_READY,       /* Not running but ready to run. */
    THREAD_BLOCKED,     /* Waiting for an event to trigger. */
    THREAD_DYING        /* About to be destroyed. */
  };

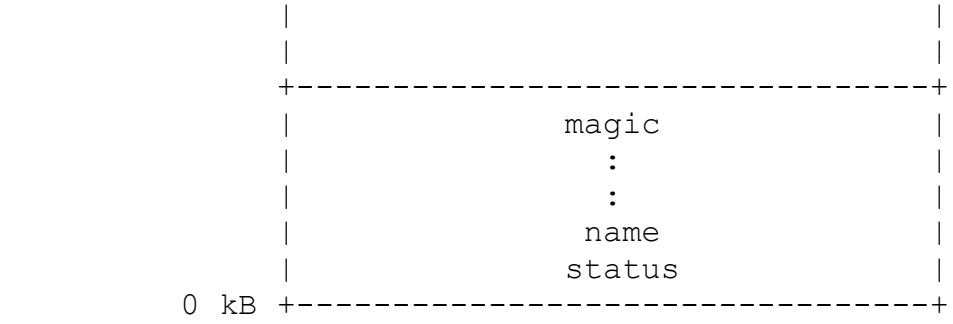
/* Thread identifier type.
   You can redefine this to whatever type you like. */
typedef int tid_t;
#define TID_ERROR ((tid_t) -1)              /* Error value for
tid_t. */

/* Thread priorities. */
#define PRI_MIN 0                       /* Lowest priority. */
#define PRI_DEFAULT 31                  /* Default priority. */
#define PRI_MAX 63                      /* Highest priority. */

/* A kernel thread or user process.

   Each thread structure is stored in its own 4 kB page.  The
   thread structure itself sits at the very bottom of the page
   (at offset 0).  The rest of the page is reserved for the
   thread's kernel stack, which grows downward from the top of
   the page (at offset 4 kB).  Here's an illustration:

        4 kB +---------------------------------+
             |          kernel stack           |
             |                |                |
             |                |                |
             |                V                |
             |         grows downward          |
             |                                 |
             |                                 |
             |                                 |
             |                                 |
             |                                 |
             |                                 |
```



The upshot of this is twofold:

1. First, `struct thread' must not be allowed to grow too big. If it does, then there will not be enough room for the kernel stack. Our base `struct thread' is only a few bytes in size. It probably should stay well under 1 kB.

2. Second, kernel stacks must not be allowed to grow too large. If a stack overflows, it will corrupt the thread state. Thus, kernel functions should not allocate large structures or arrays as non-static local variables. Use dynamic allocation with malloc() or palloc_get_page() instead.

The first symptom of either of these problems will probably be an assertion failure in thread_current(), which checks that the `magic' member of the running thread's `struct thread' is set to THREAD_MAGIC. Stack overflow will normally change this value, triggering the assertion. */

```
/* The `elem' member has a dual purpose.  It can be an element in
   the run queue (thread.c), or it can be an element in a
   semaphore wait list (synch.c).  It can be used these two ways
   only because they are mutually exclusive: only a thread in the
   ready state is on the run queue, whereas only a thread in the
   blocked state is on a semaphore wait list. */
struct thread
  {
    /* Owned by thread.c. */
    tid_t tid;                          /* Thread identifier. */
    enum thread_status status;          /* Thread state. */
    char name[16];                         /* Name (for debugging
purposes). */
```

```
    uint8_t *stack;                         /* Saved stack pointer.
*/
    int priority;                          /* Priority. */
    struct list_elem allelem;               /* List element for all
threads list. */

    /* Shared between thread.c and synch.c. */
    struct list_elem elem;                 /* List element. */

#ifdef USERPROG
    /* Owned by userprog/process.c. */
    uint32_t *pagedir;                     /* Page directory. */
    struct thread* parent;                          // Указатель на
родительский процесс
    struct list children;                       // Список дочерних
процессов
    struct child_status* child_status;   // Структура состояния
процесса
    int exit_status;                             // Код завершения
процесса
    char prog_name[64];                         // Имя исполняемой
программы
#endif

    /* Owned by thread.c. */
    unsigned magic;                              /* Detects stack
overflow. */
  };

/* If false (default), use round-robin scheduler.
   If true, use multi-level feedback queue scheduler.
   Controlled by kernel command-line option "-o mlfqs". */
extern bool thread_mlfqs;

void thread_init (void);
void thread_start (void);

void thread_tick (void);
void thread_print_stats (void);

typedef void thread_func (void *aux);
tid_t thread_create (const char *name, int priority, thread_func
*, void *);

void thread_block (void);
void thread_unblock (struct thread *);

struct thread *thread_current (void);
tid_t thread_tid (void);
const char *thread_name (void);

void thread_exit (void) NO_RETURN;
void thread_yield (void);
```

```
/* Performs some operation on thread t, given auxiliary data
AUX. */
typedef void thread_action_func (struct thread *t, void *aux);
void thread_foreach (thread_action_func *, void *);

int thread_get_priority (void);
void thread_set_priority (int);

int thread_get_nice (void);
void thread_set_nice (int);
int thread_get_recent_cpu (void);
int thread_get_load_avg (void);

#endif /* threads/thread.h */
```

Листинг Б «process.c»

```
#include "userprog/process.h"
#include <debug.h>
#include <inttypes.h>
#include <round.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "userprog/gdt.h"
#include "userprog/pagedir.h"
#include "userprog/tss.h"
#include "filesys/directory.h"
#include "filesys/file.h"
#include "filesys/filesys.h"
#include "threads/flags.h"
#include "threads/init.h"
#include "threads/interrupt.h"
#include "threads/malloc.h"
#include "threads/palloc.h"
#include "threads/synch.h"
#include "threads/thread.h"
#include "threads/vaddr.h"

static thread_func start_process NO_RETURN;
static bool load (const char *cmdline, void (**eip) (void), void
**esp);

struct child_status
{
  tid_t tid;                // ID дочернего процесса
  int exit_status;          // Код завершения ребёнка
  bool exited;              // Флаг "ребёнок уже завершился"
  bool waited;              // Флаг " на него уже вызвали wait "
  struct semaphore sema;    /* Синхронизация родителя и ребёнка,
                               Родитель    ждёт    пока    ребёнок
завершится */
  struct list_elem elem;    // Элемент списка детей
};

struct process_args
{
  char *cmdline;
  struct child_status *status;
};

static struct child_status* find_child_status(struct thread *t,
tid_t child_tid);

/* Starts a new thread running a user program loaded from
   FILENAME.   The new thread may be scheduled (and may even
exit)
   before process_execute() returns.  Returns the new process's
```

```
   thread id, or TID_ERROR if the thread cannot be created. */
tid_t
process_execute (const char *file_name)
{
  struct  process_args*  args;      //  Контейнер  для  передачи
аргументов потоку
  struct child_status* status; // Структура о состоянии ребёнка
  char *fn_copy;               // Копия командной строки
  char name_copy[64];          // Для извлечения имени программы
  char *save_ptr;                 // Указатель на позицию внутри
строки(для strtok_r)
  char *prog_name;             // Имя исполнямого файла
  tid_t tid;

  /* Копируем командную строку.
     Otherwise there's a race between the caller and load(). */
  fn_copy = palloc_get_page (0);
  if (fn_copy == NULL)
    return TID_ERROR;
  strlcpy (fn_copy, file_name, PGSIZE);

  /* Создание child_status и инициализация */
  status = malloc(sizeof *status);
  if (status==NULL)
  {
    palloc_free_page(fn_copy);
    return TID_ERROR;
  }
  status->exit_status = -1;
  status->exited = false;
  status->waited=false;
  sema_init(&status->sema,0);
  /* ================================== */

  /* Создаём структуру process_args */
  args = malloc(sizeof *args);
  if(args==NULL)
  {
    free(status);
    palloc_free_page(fn_copy);
    return TID_ERROR;
  }
  args->cmdline=fn_copy;
  args->status = status;
  /* ============================  */

  /* Извлекаем имя программы */
  strlcpy(name_copy,file_name,sizeof name_copy);
  prog_name = strtok_r(name_copy, " ", &save_ptr);
  if (prog_name==NULL)
    prog_name=name_copy;

  /* Create a new thread to execute FILE_NAME. */
```

```
  tid = thread_create (prog_name, PRI_DEFAULT, start_process,
args);
  if (tid == TID_ERROR)
  {
    free(args);
    free(status);
    palloc_free_page (fn_copy);
  }
  else
  {
    status->tid=tid;
    list_push_back(&thread_current()->children, &status->elem);
  }
  return tid;
}

/* A thread function that loads a user process and starts it
   running. */
static void
start_process (void *file_name_)
{
  struct process_args *proc_args=file_name_;        // Приводит
указатель к типу struct process_args
  char *file_name = proc_args ->cmdline;           // Извлекаем
строку командной строки
  struct child_status *status=proc_args->status;  // Извлекаем
структуру состояния
  struct thread *cur = thread_current();           // Получаем
структуру текущего потока
  struct intr_frame if_;
  bool success;


  cur->child_status = status;                      // Связываем
поток и его child_status
  char prog_copy[sizeof cur->prog_name];              // Делаем
копию для извлечения имени программы
  strlcpy (prog_copy,file_name,sizeof prog_copy);
  char* save_ptr;
  char* prog = strtok_r(prog_copy, " ", &save_ptr); // Получаем
имя программы
  if(prog!=NULL)
    strlcpy(cur->prog_name,prog,sizeof    cur->prog_name);    //
Сохраняем имя программы в thread
  else
    cur->prog_name[0]='\0';
  cur->exit_status = -1;                                 // Код
завершения по умолчанию
  free(proc_args);                                        //
Освобождаем временную структуру

  /* Initialize interrupt frame and load executable. */
  memset (&if_, 0, sizeof if_);  // Обнуление все регистров
```

```
  if_.gs = if_.fs = if_.es = if_.ds = if_.ss = SEL_UDSEG;
  if_.cs = SEL_UCSEG;
  if_.eflags = FLAG_IF | FLAG_MBS;
  // Загружаем программу в память и строим пользовательский стек
  success = load (file_name, &if_.eip, &if_.esp);

  /* If load failed, quit. */
  palloc_free_page (file_name);
  if (!success)
    thread_exit ();

  /* Start the user process by simulating a return from an
     interrupt, implemented by intr_exit (in
     threads/intr-stubs.S).  Because intr_exit takes all of its
     arguments on the stack in the form of a `struct
intr_frame',
     we just point the stack pointer (%esp) to our stack frame
     and jump to it. */
  asm volatile ("movl %0, %%esp; jmp intr_exit" : : "g" (&if_) :
"memory");
  NOT_REACHED ();
}

/* Waits for thread TID to die and returns its exit status.  If
   it was terminated by the kernel (i.e. killed due to an
   exception), returns -1.  If TID is invalid or if it was not a
   child of the calling process, or if process_wait() has
already
   been successfully called for the given TID, returns -1
   immediately, without waiting.

   This function will be implemented in problem 2-2.  For now,
it
   does nothing. */
int
process_wait (tid_t child_tid) // Родительский процесс вызывает
wait , чтобы дождаться завершения конкретного ребёнка
{
  struct            child_status            *status            =
find_child_status(thread_current(),child_tid);

  if (status == NULL || status ->waited)
    return -1;

  status->waited = true; // Мы начали ждать
  if(!status->exited)
    sema_down(&status->sema); /* Если ребёнок ещё не завершился
ждём.
    Когда он завершится, то вызовет process_exit(из thread_exit
в thread.c),
    sema_up */

  list_remove(&status->elem);
```

```
  int exit_status = status->exit_status;
  free(status);
  return exit_status;
}

/* Free the current process's resources. */
void
process_exit (void)
{
  struct thread *cur = thread_current ();
  uint32_t *pd;

  if(cur->child_status !=NULL)
  {
    cur->child_status->exit_status    =    cur->exit_status;    //
Сохраняем код завершения ребёнка
    cur->child_status->exited = true;
    sema_up(&cur->child_status->sema);
  }

  if (cur->pagedir != NULL)
    printf("%s: exit(%d)\n",cur->prog_name, cur->exit_status);

  /*  Destroy  the  current  process's  page  directory  and  switch
back
     to the kernel-only page directory. */
  pd = cur->pagedir;
  if (pd != NULL)
    {
      /* Correct ordering here is crucial.  We must set
         cur->pagedir to NULL before switching page directories,
         so that a timer interrupt can't switch back to the
         process page directory.  We must activate the base page
         directory before destroying the process's page
         directory, or our active page directory will be one
         that's been freed (and cleared). */
      cur->pagedir = NULL;
      pagedir_activate (NULL);
      pagedir_destroy (pd);
    }
}

/* Sets up the CPU for running user code in the current
   thread.
   This function is called on every context switch. */
void
process_activate (void)
{
  struct thread *t = thread_current ();

  /* Activate thread's page tables. */
  pagedir_activate (t->pagedir);
```

```
  /* Set thread's kernel stack for use in processing
     interrupts. */
  tss_update ();
}
```

```
/* We load ELF binaries.  The following definitions are taken
   from the ELF specification, [ELF1], more-or-less verbatim.
*/

/* ELF types.  See [ELF1] 1-2. */
typedef uint32_t Elf32_Word, Elf32_Addr, Elf32_Off;
typedef uint16_t Elf32_Half;

/* For use with ELF types in printf(). */
#define PE32Wx PRIx32   /* Print Elf32_Word in hexadecimal. */
#define PE32Ax PRIx32   /* Print Elf32_Addr in hexadecimal. */
#define PE32Ox PRIx32   /* Print Elf32_Off in hexadecimal. */
#define PE32Hx PRIx16   /* Print Elf32_Half in hexadecimal. */

/* Executable header.  See [ELF1] 1-4 to 1-8.
   This appears at the very beginning of an ELF binary. */
struct Elf32_Ehdr
  {
    unsigned char e_ident[16];
    Elf32_Half    e_type;
    Elf32_Half    e_machine;
    Elf32_Word    e_version;
    Elf32_Addr    e_entry;
    Elf32_Off     e_phoff;
    Elf32_Off     e_shoff;
    Elf32_Word    e_flags;
    Elf32_Half    e_ehsize;
    Elf32_Half    e_phentsize;
    Elf32_Half    e_phnum;
    Elf32_Half    e_shentsize;
    Elf32_Half    e_shnum;
    Elf32_Half    e_shstrndx;
  };

/* Program header.  See [ELF1] 2-2 to 2-4.
   There are e_phnum of these, starting at file offset e_phoff
   (see [ELF1] 1-6). */
struct Elf32_Phdr
  {
    Elf32_Word p_type;
    Elf32_Off  p_offset;
    Elf32_Addr p_vaddr;
    Elf32_Addr p_paddr;
    Elf32_Word p_filesz;
    Elf32_Word p_memsz;
    Elf32_Word p_flags;
    Elf32_Word p_align;
  };

/* Values for p_type.  See [ELF1] 2-3. */
#define PT_NULL    0            /* Ignore. */
#define PT_LOAD    1            /* Loadable segment. */
#define PT_DYNAMIC 2            /* Dynamic linking info. */
```

```
#define PT_INTERP  3            /* Name of dynamic loader. */
#define PT_NOTE    4            /* Auxiliary info. */
#define PT_SHLIB   5            /* Reserved. */
#define PT_PHDR    6            /* Program header table. */
#define PT_STACK   0x6474e551   /* Stack segment. */

/* Flags for p_flags.  See [ELF3] 2-3 and 2-4. */
#define PF_X 1          /* Executable. */
#define PF_W 2          /* Writable. */
#define PF_R 4          /* Readable. */

static bool setup_stack (void **esp);
static bool validate_segment (const struct Elf32_Phdr *, struct
file *);
static bool load_segment (struct file *file, off_t ofs, uint8_t
*upage,
                          uint32_t      read_bytes,      uint32_t
zero_bytes,
                          bool writable);

static struct child_status *find_child_status(struct thread *t,
tid_t child_tid)
{
  struct list_elem *e;

  for(e=list_begin(&t->children);e!=list_end(&t-
>children);e=list_next(e))
  {
    struct        child_status        *child=list_entry(e,struct
child_status,elem);
    if (child->tid == child_tid)
      return child;
  }
  return NULL;
}


/* Loads an ELF executable from FILE_NAME into the current
thread.
   Stores the executable's entry point into *EIP
   and its initial stack pointer into *ESP.
   Returns true if successful, false otherwise. */
bool
load (const char *file_name, void (**eip) (void), void **esp)
{
  struct thread *t = thread_current ();
  struct Elf32_Ehdr ehdr;
  struct file *file = NULL;
  off_t file_ofs;
  bool success = false;
  char *file_name_copy = NULL; // Для имени программы
  char *cmdline_copy = NULL;   // для argv[]
  char *argv[64];              // Массив указателей на аргументы
```

```
  uint32_t   argv_addresses[64];   //   Адреса   аргументов   в
пользовательском стеке
  int argc=0;                  // Кол-во аргументов
  int i;

  cmdline_copy = palloc_get_page(0);
  file_name_copy = palloc_get_page(0);
  if(cmdline_copy ==NULL || file_name_copy == NULL)
    goto done;

  strlcpy(cmdline_copy,file_name,PGSIZE);
  strlcpy(file_name_copy,file_name,PGSIZE);

  char *save_ptr;
  char *prog_name = strtok_r(file_name_copy, " ", &save_ptr);
  if(prog_name==NULL)
    goto done;

  char *token;
  char *arg_save_ptr;
  token=strtok_r(cmdline_copy, " ",&arg_save_ptr);
  while(token !=NULL)
  {
    if(argc>=64)
      goto done;

    argv[argc++] = token;

    token = strtok_r(NULL, " ", &arg_save_ptr);
  }
  /* Allocate and activate page directory. */
  t->pagedir = pagedir_create ();
  if (t->pagedir == NULL)
    goto done;
  process_activate ();

  /* Open executable file. */
  file = filesys_open (prog_name);
  if (file == NULL)
    {
      printf ("load: %s: open failed\n", prog_name);
      goto done;
    }

  /* Read and verify executable header. */
  if (file_read (file, &ehdr, sizeof ehdr) != sizeof ehdr
      || memcmp (ehdr.e_ident, "\177ELF\1\1\1", 7)
      || ehdr.e_type != 2
      || ehdr.e_machine != 3
      || ehdr.e_version != 1
      || ehdr.e_phentsize != sizeof (struct Elf32_Phdr)
      || ehdr.e_phnum > 1024)
    {
```

```
      printf ("load: %s: error loading executable\n",
file_name);
      goto done;
    }

  /* Read program headers. */
  file_ofs = ehdr.e_phoff;
  for (i = 0; i < ehdr.e_phnum; i++)
    {
      struct Elf32_Phdr phdr;

      if (file_ofs < 0 || file_ofs > file_length (file))
        goto done;
      file_seek (file, file_ofs);

      if (file_read (file, &phdr, sizeof phdr) != sizeof phdr)
        goto done;
      file_ofs += sizeof phdr;
      switch (phdr.p_type)
        {
        case PT_NULL:
        case PT_NOTE:
        case PT_PHDR:
        case PT_STACK:
        default:
          /* Ignore this segment. */
          break;
        case PT_DYNAMIC:
        case PT_INTERP:
        case PT_SHLIB:
          goto done;
        case PT_LOAD:
          if (validate_segment (&phdr, file))
            {
              bool writable = (phdr.p_flags & PF_W) != 0;
              uint32_t file_page = phdr.p_offset & ~PGMASK;
              uint32_t mem_page = phdr.p_vaddr & ~PGMASK;
              uint32_t page_offset = phdr.p_vaddr & PGMASK;
              uint32_t read_bytes, zero_bytes;
              if (phdr.p_filesz > 0)
                {
                  /* Normal segment.
                     Read initial part from disk and zero the
rest. */
                  read_bytes = page_offset + phdr.p_filesz;
                  zero_bytes = (ROUND_UP (page_offset +
phdr.p_memsz, PGSIZE)
                                - read_bytes);
                }
              else
                {
                  /* Entirely zero.
                     Don't read anything from disk. */
```

```
                  read_bytes = 0;
                  zero_bytes    =    ROUND_UP    (page_offset    +
phdr.p_memsz, PGSIZE);
                }
              if   (!load_segment   (file,   file_page,   (void   *)
mem_page,
                                     read_bytes,          zero_bytes,
writable))
              {
                goto done;
              }

            }
          else
            goto done;
          break;
        }
    }

  /* Set up stack. */
  if (!setup_stack (esp))
    goto done;

  uint8_t *sp = *esp;  // Вершина стека
  for(i  =  argc  -1;  i>=0;  i--)  // Сначала кладём в стек сами
аргументы
  {
    size_t len = strlen(argv[i])+1;
    sp -= len;
    memcpy (sp,argv[i],len);
    argv_addresses[i] = (uint32_t) sp;
  }

  size_t  padding  =  (size_t)  sp%4;  // Делаем выравнивание для
массивов указателей (4 byte)
  if (padding!=0)
  {
    sp -= padding;
    memset(sp,0,padding);
  }

  sp -=sizeof  (char*); // Отступаем 4 байта, для конца массива
указателей
  *(char **) sp = NULL;

  for(i = argc -1; i>=0; i--)
  {
    sp -= sizeof(char *);
    *(char  **)  sp  =  (char  *)  argv_addresses[i];  //  Записываем
адреса указателей в стек
  }
```

```
  char **argv_start = (char **) sp; // Указатель на начало
массива
  sp -= sizeof (char **);           // Сдвигаемся на 4 байта
  *(char ***) sp = argv_start;       // Записываем указатель на
начало массива

  sp-=sizeof(int);    //Кладём argc
  *(int *) sp = argc;

  sp-= sizeof(void *); // Кладём фейковый return address
  *(void **) sp =NULL;

  *esp = sp;

  /* Start address. */
  *eip = (void (*) (void)) ehdr.e_entry;

  success = true;

 done:
  /* We arrive here whether the load is successful or not. */
  if (cmdline_copy!=NULL)
    palloc_free_page(cmdline_copy);
  if (file_name_copy !=NULL)
    palloc_free_page(file_name_copy);
  if (file !=NULL)
    file_close(file);
  return success;
}
```

```
/* load() helpers. */

static bool install_page (void *upage, void *kpage, bool
writable);

/* Checks whether PHDR describes a valid, loadable segment in
   FILE and returns true if so, false otherwise. */
static bool
validate_segment (const struct Elf32_Phdr *phdr, struct file
*file)
{
  /* p_offset and p_vaddr must have the same page offset. */
  if ((phdr->p_offset & PGMASK) != (phdr->p_vaddr & PGMASK))
    return false;

  /* p_offset must point within FILE. */
  if (phdr->p_offset > (Elf32_Off) file_length (file))
    return false;

  /* p_memsz must be at least as big as p_filesz. */
  if (phdr->p_memsz < phdr->p_filesz)
    return false;

  /* The segment must not be empty. */
  if (phdr->p_memsz == 0)
    return false;

  /* The virtual memory region must both start and end within
the
     user address space range. */
  if (!is_user_vaddr ((void *) phdr->p_vaddr))
    return false;
  if (!is_user_vaddr ((void *) (phdr->p_vaddr + phdr->p_memsz)))
    return false;

  /* The region cannot "wrap around" across the kernel virtual
     address space. */
  if (phdr->p_vaddr + phdr->p_memsz < phdr->p_vaddr)
    return false;

  /* Disallow mapping page 0.
     Not only is it a bad idea to map page 0, but if we allowed
     it then user code that passed a null pointer to system
calls
     could quite likely panic the kernel by way of null pointer
     assertions in memcpy(), etc. */
  if (phdr->p_vaddr < PGSIZE)
    return false;

  /* It's okay. */
  return true;
}
```

```
/* Loads a segment starting at offset OFS in FILE at address
   UPAGE.  In total, READ_BYTES + ZERO_BYTES bytes of virtual
   memory are initialized, as follows:

        - READ_BYTES bytes at UPAGE must be read from FILE
          starting at offset OFS.

        - ZERO_BYTES bytes at UPAGE + READ_BYTES must be zeroed.

   The pages initialized by this function must be writable by
the
   user process if WRITABLE is true, read-only otherwise.

   Return true if successful, false if a memory allocation error
   or disk read error occurs. */
static bool
load_segment (struct file *file, off_t ofs, uint8_t *upage,
              uint32_t  read_bytes,  uint32_t  zero_bytes,  bool
writable)
{
  ASSERT ((read_bytes + zero_bytes) % PGSIZE == 0);
  ASSERT (pg_ofs (upage) == 0);
  ASSERT (ofs % PGSIZE == 0);

  file_seek (file, ofs);
  while (read_bytes > 0 || zero_bytes > 0)
    {
      /* Calculate how to fill this page.
         We will read PAGE_READ_BYTES bytes from FILE
         and zero the final PAGE_ZERO_BYTES bytes. */
      size_t page_read_bytes = read_bytes < PGSIZE ? read_bytes
: PGSIZE;
      size_t page_zero_bytes = PGSIZE - page_read_bytes;

      /* Get a page of memory. */
      uint8_t *kpage = palloc_get_page (PAL_USER);
      if (kpage == NULL)
        return false;

      /* Load this page. */
      if  (file_read  (file,  kpage,  page_read_bytes)  !=  (int)
page_read_bytes)
        {
          palloc_free_page (kpage);
          return false;
        }
      memset (kpage + page_read_bytes, 0, page_zero_bytes);

      /* Add the page to the process's address space. */
      if (!install_page (upage, kpage, writable))
        {
          palloc_free_page (kpage);
          return false;
```

```
        }

      /* Advance. */
      read_bytes -= page_read_bytes;
      zero_bytes -= page_zero_bytes;
      upage += PGSIZE;
    }
  return true;
}

/* Create a minimal stack by mapping a zeroed page at the top of
   user virtual memory. */
static bool
setup_stack (void **esp)
{
  uint8_t *kpage;
  bool success = false;

  kpage = palloc_get_page (PAL_USER | PAL_ZERO);
  if (kpage != NULL)
    {
      success = install_page (((uint8_t *) PHYS_BASE) - PGSIZE,
kpage, true);
      if (success)
        *esp = PHYS_BASE;
      else
        palloc_free_page (kpage);
    }
  return success;
}

/* Adds a mapping from user virtual address UPAGE to kernel
   virtual address KPAGE to the page table.
   If WRITABLE is true, the user process may modify the page;
   otherwise, it is read-only.
   UPAGE must not already be mapped.
   KPAGE should probably be a page obtained from the user pool
   with palloc_get_page().
   Returns true on success, false if UPAGE is already mapped or
   if memory allocation fails. */
static bool
install_page (void *upage, void *kpage, bool writable)
{
  struct thread *t = thread_current ();

  /* Verify that there's not already a page at that virtual
     address, then map our page there. */
  return (pagedir_get_page (t->pagedir, upage) == NULL
          &&   pagedir_set_page   (t->pagedir,   upage,   kpage,
writable));
```

Листинг В «syscall.c»

```
#include "userprog/syscall.h"
#include <debug.h>
#include <stdio.h>
#include <stdint.h>
#include <syscall-nr.h>
#include "threads/interrupt.h"
#include "threads/thread.h"
#include "threads/vaddr.h"
#include "userprog/pagedir.h"
#include "userprog/process.h"

static void syscall_handler (struct intr_frame *);
static void validate_user_pointer(const void *uaddr);
static void sys_exit(int statis) NO_RETURN;

void
syscall_init (void)
{
  intr_register_int    (0x30,    3,    INTR_ON,    syscall_handler,
"syscall");
}

static void
syscall_handler (struct intr_frame *f)
{
  uint32_t *usp = (uint32_t *) f->esp;

  validate_user_pointer(usp);
  int syscall_number = usp[0];

  switch(syscall_number)
  {
  case SYS_EXIT:
    validate_user_pointer(usp+1);
    sys_exit((int)usp[1]);
    break;
  case SYS_WRITE: //write(fd(stdin-0,stdout-1), buffer, size)
    validate_user_pointer(usp+1);
    validate_user_pointer(usp+2);
    validate_user_pointer(usp+3);
    {
      int fd = (int) usp[1];
      const void *buffer = (const void *) usp[2];
      unsigned size = (unsigned) usp[3];

      if (size>0)
      {
        validate_user_pointer((const   uint8_t   *)   buffer);   //
проверяем начало буфера
        validate_user_pointer((const  uint8_t  *)  buffer  +size  -
1); // проверяем конец буфера
```

```
      }

      if (fd==1)
      {
        putbuf(buffer,size);
        f->eax = (int) size;
      }
      else
        f->eax = -1;
    }
    break;
  default:
    sys_exit(-1);
    break;
  }
}

static void validate_user_pointer(const void *uaddr)
{
  if (uaddr == NULL || !is_user_vaddr(uaddr)
      ||       pagedir_get_page       (thread_current()->pagedir,
uaddr)==NULL)
  {
    sys_exit(-1);
  }
}

static void sys_exit(int status)
{
  thread_current()->exit_status = status;
  thread_exit();
}
```

Листинг В «svconv.c»

```
#include <ctype.h>
#include <stdbool.h>
#include <stdio.h>
#include <syscall.h>

/* Сравнение строк без учёта регистра*/
static bool
iequals (const char *a, const char *b)
{
  for (; *a != '\0' && *b != '\0'; a++, b++)
    {
      if (tolower (*a) != tolower (*b))
        return false;
    }
  return *a == '\0' && *b == '\0';
}

static int
parse_base (const char *arg)
{
  if (iequals (arg, "2") || iequals (arg, "bin") || iequals
(arg, "binary"))
    return 2;
  if (iequals (arg, "8") || iequals (arg, "oct") || iequals
(arg, "octal"))
    return 8;
  if (iequals (arg, "10") || iequals (arg, "dec") || iequals
(arg, "decimal"))
    return 10;
  if (iequals (arg, "16") || iequals (arg, "hex") || iequals
(arg, "hexadecimal"))
    return 16;
  return -1;
}

/* Символ -> Цифра */
static int
digit_value (char c)
{
  if (c >= '0' && c <= '9')
    return c - '0';
  if (c >= 'A' && c <= 'F')
    return 10 + (c - 'A');
  if (c >= 'a' && c <= 'f')
    return 10 + (c - 'a');
  return -1;
}

/* Строка -> int */
static bool
parse_number (const char *str, int base, int *out)
```

```
{
  int result = 0;
  bool neg = false;

  if (*str == '+' || *str == '-')
    {
      neg = *str == '-';
      str++;
    }

  if (*str == '\0')
    return false;

  while (*str != '\0')
    {
      int d = digit_value (*str);
      if (d < 0 || d >= base)
        return false;
      result = result * base + d;
      str++;
    }

  if (neg)
    result = -result;
  *out = result;
  return true;
}

/* int -> string
   Переводит число в строку в нужном основании */
static void
to_base_string (int value, int base, char *buffer, size_t
buf_size)
{
  static const char digits[] = "0123456789ABCDEF";
  char tmp[35];
  size_t idx = 0;
  bool neg = value < 0;
  unsigned int v = neg ? - (unsigned int) value : (unsigned int)
value;

  if (v == 0)
    tmp[idx++] = '0';

  while (v > 0 && idx < sizeof tmp)
    {
      tmp[idx++] = digits[v % (unsigned int) base];
      v /= (unsigned int) base;
    }

  size_t out_idx = 0;
  if (neg && out_idx < buf_size - 1)
    buffer[out_idx++] = '-';
```

```
  while (idx > 0 && out_idx < buf_size - 1)
    buffer[out_idx++] = tmp[--idx];

  buffer[out_idx] = '\0';
}

static void
usage (void)
{
  printf ("Usage: snconv <number> <source base> <target
base>\n");
  printf ("Bases: 2|bin|binary, 8|oct|octal, 10|dec|decimal,
16|hex|hexadecimal\n");
}

// snconv <число> <исходное_основание> <целевое_основание>
int
main (int argc, const char *argv[])
{
  if (argc != 4)
    {
      usage ();
      return EXIT_FAILURE;
    }

  int src_base = parse_base (argv[2]);
  int dst_base = parse_base (argv[3]);
  if (src_base < 0 || dst_base < 0)
    {
      usage ();
      return EXIT_FAILURE;
    }

  int value;
  if (!parse_number (argv[1], src_base, &value))
    {
      printf ("Invalid number for base %d\n", src_base);
      return EXIT_FAILURE;
    }

  char buffer[40];
  to_base_string (value, dst_base, buffer, sizeof buffer);
  printf ("%s\n", buffer);
  return EXIT_SUCCESS;
}
```